

Social Analysis Framework - Code Skeleton

Ontological Generalization Framework

Code Structure Documentation

This document provides a comprehensive skeleton overview of each Python module in the framework, including class structures, major functions, and module responsibilities.

main.py - Main Orchestration Script

Classes:

```
OntologicalExperimentPipeline
    __init__(config: Dict)
    setup_data()
    define_ontological_space() -> List[Dict]
    run_experiment(exp_config: Dict) -> Dict
    run_all_experiments()
    analyze_results()
    run()
```

Functions:

```
parse_args()
main()
```

Purpose:

Orchestrates the end-to-end pipeline for running ontological experiments across TAG variants. This file manages the complete workflow from data loading, through experiment execution across different ontological configurations, to final meta-analysis and result visualization. It serves as the entry point for the entire framework and coordinates all other modules.

Social Analysis Framework - Code Skeleton

analyzer.py - Meta-Analysis Module

Classes:

PerformanceBand

```
    categorize_kl(kl: float) -> str (static)
    categorize_accuracy(acc: float) -> str (static)
    categorize_metric(metric_name: str, value: float) -> str (static)
```

DecisionTreeAnalyzer

```
    __init__()
    add_experiment(task_idx, node_idx, edge_idx, text_idx, test_metrics)
    build_decision_tree_table() -> pd.DataFrame
    train_decision_tree(...) -> Dict
    plot_decision_tree(...)
    predict_performance(task_idx, node_idx, edge_idx, text_idx) -> str
    analyze_best_configurations(top_k: int) -> pd.DataFrame
    analyze_by_ontology() -> Dict
    save_analysis(output_dir: str)
    load_analysis(input_dir: str)
```

Purpose:

Performs meta-analysis across TAG experiments using decision tree models to identify patterns in ontological indices. This module tracks all experiment results, builds decision tree classifiers to predict performance based on ontological configurations, and provides comprehensive analysis of which combination of task/node/edge/text indices yield optimal performance. It generates visualizations and exports analysis results for interpretation.

Social Analysis Framework - Code Skeleton

constructor.py - TAG Constructor Module

Classes:

TAGConfiguration

Attributes: TASKS, NODES, EDGES, SIMILARITY_THRESHOLDS, TEXT_FIDELITIES

TAG

```
__init__(task_idx, node_idx, edge_idx, text_idx, graph)
get_identifier() -> str
to_pyg_data(node_list, embeddings, labels, device) -> Data
```

TAGBuilder

```
__init__(data_manager)
build_coauthorship_graph(node_type) -> nx.Graph
build_similarity_graph(embeddings, node_list, threshold, k, node_type) -> nx.Graph
build(task_idx, node_idx, edge_idx, text_idx, similarity_threshold) -> TAG
save_tag(tag: TAG, output_dir: str)
load_tag(filepath: str) -> TAG
```

Purpose:

Implements the Builder pattern for constructing Text-Augmented Graphs (TAGs) based on ontological indices. This module defines the complete ontology space and builds different graph structures (coauthorship, similarity-based) with various node types and edge types, then converts them to PyTorch Geometric format for GNN training. It encapsulates all the logic for creating TAG variants that represent different perspectives on the underlying academic data.

Social Analysis Framework - Code Skeleton

data_manager.py - Data Backbone Module

Classes:

```
ArxivDataManager
__init__(data_path: str, subset_size: Optional[int])
load_data() -> pd.DataFrame
_parse_authors_field(val) (static)
handle_author(author_parsed_instance: List) -> str (static)
extract_unique_authors() -> set
generate_text_fidelity_variant(text: str, fidelity: str) -> str
compute_embeddings(model_name, text_fidelity, force_recompute)
    -> Tuple[np.ndarray, np.ndarray]
save_embeddings(output_dir: str, prefix: str)
load_embeddings(input_dir: str, prefix: str)
get_category_labels(num_classes: int) -> np.ndarray
get_author_category_labels(num_classes: int) -> np.ndarray
```

Purpose:

Manages the ArXiv dataset backbone including loading, preprocessing, subsetting, and embedding generation. This module handles all data operations including parsing JSON data, extracting unique authors, computing SentenceTransformer embeddings with various text fidelity levels, and generating both paper and author category labels. It provides a clean interface for accessing processed data needed by other modules.

Social Analysis Framework - Code Skeleton

models.py - GNN Models Module

Classes:

```
SAGEPredictor (nn.Module)
    __init__(in_dim, hidden_dim, out_dim, dropout)
    forward(x, edge_index)
```

```
GCNPredictor (nn.Module)
    __init__(in_dim, hidden_dim, out_dim, dropout)
    forward(x, edge_index)
```

```
GATPredictor (nn.Module)
    __init__(in_dim, hidden_dim, out_dim, dropout, num_heads)
    forward(x, edge_index)
```

```
ModelFactory
    create_model(model_type, in_dim, hidden_dim, out_dim, dropout, **kwargs)
        (classmethod)
    register_model(name: str, model_class) (classmethod)
```

Purpose:

Defines Graph Neural Network architectures (GraphSAGE, GCN, GAT) for node classification on TAG objects. This module implements multiple GNN variants with consistent interfaces, using batch normalization and dropout for regularization, and provides a factory pattern for dynamic model instantiation. All models output log-probabilities for KL divergence-based training.

Social Analysis Framework - Code Skeleton

trainer.py - Execution Engine Module

Classes:

GNNTrainer

```
__init__(model, device, lr, weight_decay, epochs, patience)
train_one_epoch(data) -> float
evaluate(data, mask, num_classes) -> Dict[str, float]
train(data, num_classes, verbose) -> Dict[str, float]
save_model(filepath: str)
load_model(filepath: str)
```

TrainingPipeline

```
__init__(model_factory, device)
run_experiment(tag, embeddings, labels, node_list, num_classes,
              model_config, trainer_config, save_model_path,
              save_training_history) -> Dict
```

PerformanceTracker

```
__init__()
add_result(result: Dict)
get_summary() -> Dict
save_results(filepath: str)
```

Purpose:

Handles GNN model training, evaluation, and performance tracking with multiple metrics (KL divergence, top-k accuracy, cosine similarity). This module implements the training loop with early stopping, tracks training history for each epoch, and manages the complete experiment workflow from TAG to trained model. It provides a high-level interface for running experiments and collecting comprehensive performance metrics.

Social Analysis Framework - Code Skeleton

Module Dependencies and Workflow

1. `main.py` orchestrates the entire pipeline, calling all other modules in sequence.
2. `data_manager.py` loads and preprocesses the ArXiv dataset, generating embeddings.
3. `constructor.py` builds TAG variants based on ontological indices using the data manager.
4. `models.py` provides GNN architectures for node classification.
5. `trainer.py` executes training experiments on TAG objects using the GNN models.
6. `analyzer.py` performs meta-analysis on experiment results and identifies optimal configurations.

Generated: February 27, 2026